BUILDING A HIGHLY AVAILABLE KUBERNETES CLUSTER

A complete, step-by-step guide to designing, initializing, and operating a production-grade multi-master Kubernetes cluster with 3 Control Plane nodes, 2 Worker nodes, and a Load Balancer — from first principles to full deployment.

KUBERNETES | HIGH AVAILABILITY | KUBEADM

WHY THIS MATTERS

THE SINGLE-MASTER PROBLEM

In a single-master Kubernetes setup, the control plane lives entirely on one node. That node runs the API Server, Scheduler, Controller Manager, and etcd. Every worker node communicates with it. Every `kubectl` command hits it. Every cluster decision passes through it.

This creates a single point of failure. If that one master node goes down — due to hardware failure, OS crash, or a network partition — the entire cluster control plane becomes unreachable. Running pods may continue briefly, but no new workloads can be scheduled, no configuration changes can be applied, and no healing or scaling can happen.

For production systems, this is unacceptable. The solution is a **Highly Available (HA) Control Plane**: multiple master nodes working together, backed by a distributed etcd cluster, fronted by a Load Balancer.

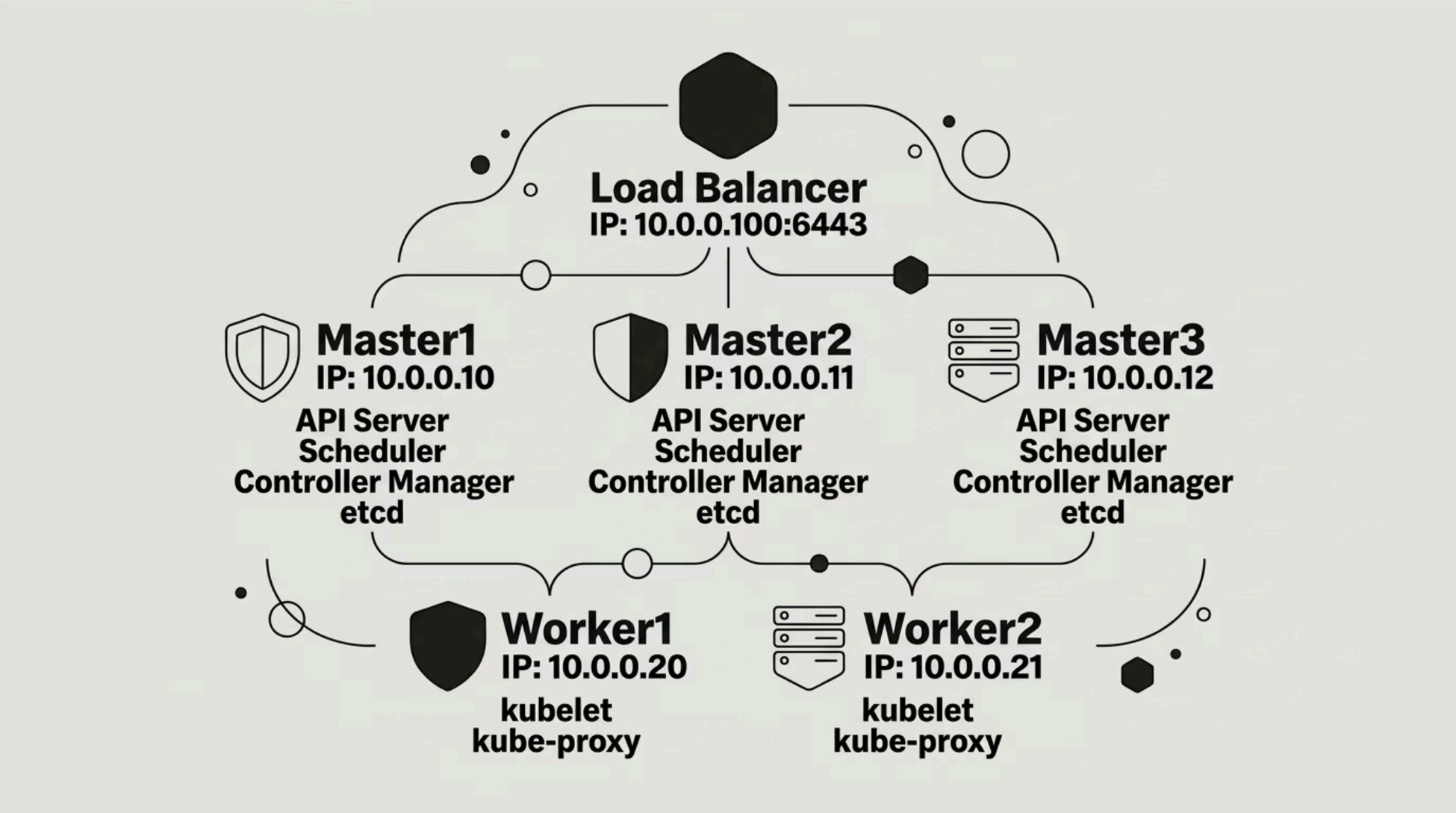
WHAT BREAKS WHEN MASTER FAILS

- `kubectl` becomes unreachable
- New pods cannot be scheduled
- Deployments cannot scale
- Nodes cannot join the cluster
- Health checks stop updating
- Secrets and ConfigMaps are inaccessible

TARGET ARCHITECTURE

WHAT WE ARE BUILDING

Our cluster consists of six machines: three Control Plane (master) nodes, two Worker nodes, and a Load Balancer that sits in front of all three API Servers. Workers never connect to individual masters — they always go through the Load Balancer.



All three masters share state through a distributed etcd cluster using the RAFT consensus algorithm. The Load Balancer distributes API requests across all healthy masters — ensuring the cluster survives the loss of any single control plane node.

CORE ARCHITECTURE PRINCIPLE

MASTERS NEVER TALK DIRECTLY TO EACH OTHER

This is the most important concept to understand before diving into the steps. Unlike a traditional database cluster or a peer-to-peer network, Kubernetes master nodes do **not** communicate directly with one another. Instead, all shared state flows through a single medium: **etcd**.

ETCD = THE SHARED BRAIN

Every cluster object — pods, nodes, deployments, secrets — is stored in etcd. All masters read and write to the same etcd cluster, so they always see the same state.

API SERVER = THE ONLY GATEWAY

No component ever reads etcd directly except the API Server. The Scheduler, Controller Manager, and kubelet all interact with etcd exclusively through the API Server.

LOAD BALANCER = STABLE ENDPOINT

Workers and operators connect to a single, stable IP. The Load Balancer routes each request to a healthy master. This decouples clients from individual master IPs entirely.

CHAPTER 1: PREPARATION

STEP 1 — PREPARE ALL FIVE MACHINES

Before any Kubernetes component is installed, every node — all three masters and both workers — must be prepared identically. Kubernetes requires swap to be completely disabled, specific kernel modules loaded, and proper network bridge parameters set in the kernel.

DISABLE SWAP (ALL NODES)

Kubernetes assumes consistent memory availability. Swap defeats its memory management. Run on every node:

```
sudo swapoff -a
sudo sed -i '/ swap / s/^/#/' /etc/fstab
```

ENABLE KERNEL MODULES

```
sudo modprobe overlay
sudo modprobe br_netfilter
```

SET SYSCTL PARAMETERS

```
cat <<EOF | sudo tee \
 /etc/sysctl.d/kubernetes.conf
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
EOF

sudo sysctl --system
```

CHAPTER 1: PREPARATION

STEP 2 — INSTALL CONTAINERD (ALL NODES)

Kubernetes needs a container runtime to pull images and run containers. **containerd** is the standard choice and is supported on all major Linux distributions. Install it on every node — masters and workers alike.

INSTALL AND CONFIGURE

```
sudo apt install -y containerd

sudo mkdir -p /etc/containerd
containerd config default | \
  sudo tee /etc/containerd/config.toml
```

The default config sets up namespaces, snapshotter, and runtime defaults correctly for Kubernetes. No manual editing is typically needed for a lab cluster.

ENABLE AND VERIFY

```
sudo systemctl restart containerd
sudo systemctl enable containerd

# Verify it is running
systemctl status containerd
```

Confirm containerd shows `active (running)`. If it fails to start, check the config file for syntax errors by running `containerd --config /etc/containerd/config.toml`.

STEP 3 — INSTALL KUBEADM, KUBELET, KUBECTL (ALL NODES)

These three binaries are the foundation of every Kubernetes node. Install the same version on every machine to avoid compatibility issues. After installation, lock the versions to prevent accidental upgrades that could destabilize the cluster.

ADD KUBERNETES REPOSITORY

```
sudo apt install -y \
  apt-transport-https curl

curl -fsSL https://pkgs.k8s.io/\
core:/stable:/v1.29/deb/Release.key \
| sudo gpg --dearmor \
-o /etc/apt/keyrings/kubernetes.gpg

echo 'deb [signed-by=/etc/apt/\
keyrings/kubernetes.gpg] \
https://pkgs.k8s.io/core:/stable:\
/v1.29/deb/ /' | sudo tee \
/etc/apt/sources.list.d/kubernetes.list
```

INSTALL AND LOCK

```
sudo apt update
sudo apt install -y \
  kubelet kubeadm kubectl

# Lock versions — very important
sudo apt-mark hold \
  kubelet kubeadm kubectl
```

kubelet runs on every node and manages containers. **kubeadm** bootstraps the cluster. **kubectl** is the command-line client — needed primarily on master nodes.

STEP 4 — SET UP THE LOAD BALANCER

The Load Balancer is the stable entry point into the cluster's API. Every worker node, every `kubectl` command, and every joining node will use this single IP and port — **10.0.0.100:6443**. The LB's only job is to forward TCP traffic to any healthy master node.

HAPROXY CONFIGURATION

```
sudo apt install haproxy
sudo nano /etc/haproxy/haproxy.cfg
```

```
frontend kubernetes
  bind 10.0.0.100:6443
  mode tcp
  option tcplog
  default_backend kubernetes-masters

backend kubernetes-masters
  mode tcp
  balance roundrobin
  server master1 10.0.0.10:6443 check
  server master2 10.0.0.11:6443 check
  server master3 10.0.0.12:6443 check
```

```
sudo systemctl restart haproxy
```

ON AWS (EC2) — USE NLB INSTEAD

If your cluster runs on EC2, skip HAProxy and create an AWS **Network Load Balancer** (NLB):

- Protocol: **TCP**, Port: **6443**
- Scheme: **Internal**
- Target Group: instances, TCP 6443
- Register all 3 master EC2 instances
- Health Check: TCP port 6443

ALB does **not** work here — the Kubernetes API uses raw TCP, not HTTP/HTTPS at the LB layer. Copy the NLB DNS name; you will use it as your `--control-plane-endpoint`.

STEP 5 — INITIALIZE THE FIRST MASTER NODE

Only **one** master node is ever initialized. Master1 bootstraps the entire cluster, generates all certificates, starts etcd, launches the API Server, Scheduler, and Controller Manager, and then prints the join commands for every other node. This is the most critical command in the entire process.

```
sudo kubeadm init \
  --control-plane-endpoint "10.0.0.100:6443" \
  --upload-certs \
  --pod-network-cidr=192.168.0.0/16
```

1

PREFLIGHT CHECKS

Verifies swap is off, ports are free, container runtime is running, and kernel modules are loaded.

2

TLS CERTIFICATES GENERATED

CA, API Server, etcd, front-proxy, and admin certificates written to `/etc/kubernetes/pki/`.

3

KUBECONFIG FILES CREATED

Config files for admin, controller-manager, scheduler, and kubelet written to `/etc/kubernetes/`.

4

STATIC PODS STARTED

etcd, kube-apiserver, kube-controller-manager, and kube-scheduler manifests written to `/etc/kubernetes/manifests/`. Kubelet starts them immediately.

5

BOOTSTRAP TOKEN PRINTED

A worker join command and a control-plane join command are printed to the terminal. Save both immediately.

WHAT KUBEADM INIT PRODUCES

At the end of a successful `kubeadm init`, the terminal prints two distinct join commands. Save both — they expire after 24 hours by default. The certificate key (used only for master joins) is especially sensitive: it encrypts the PKI material shared with joining control-plane nodes.

WORKER JOIN COMMAND

```
kubeadm join 10.0.0.100:6443 \
  --token abcdef.123456789 \
  --discovery-token-ca-cert-hash \
  sha256:xxxxxxxxxxxxxxxx
```

Use this on Worker1 and Worker2. No additional flags required.

CONTROL PLANE JOIN COMMAND

```
kubeadm join 10.0.0.100:6443 \
  --token abcdef.123456789 \
  --discovery-token-ca-cert-hash \
  sha256:xxxxxxxxxxxxxxxx \
  --control-plane \
  --certificate-key abcd1234efgh
```

Use this on Master2 and Master3. The `--control-plane` flag and `--certificate-key` are what differentiate a master join from a worker join.

If you lose the join command, regenerate it on Master1: run `kubeadm token create --print-join-command` for the worker command, then run `kubeadm init phase upload-certs --upload-certs` to get a fresh certificate key and append `--control-plane --certificate-key <key>` for masters.

STEP 6 — CONFIGURE KUBECTL ON MASTER1

After initialization, the `admin.conf` file on Master1 contains the credentials needed to talk to the cluster. Copy it to the standard `~/.kube/config` location so `kubectl` works without extra flags.

CONFIGURE KUBECTL ACCESS

```
mkdir -p $HOME/.kube
sudo cp /etc/kubernetes/admin.conf \
  $HOME/.kube/config
sudo chown $(id -u):$(id -g) \
  $HOME/.kube/config
```

VERIFY THE CLUSTER EXISTS

```
# Should show master1 as NotReady
# (network plugin not yet installed)
kubectl get nodes

# Confirm system pods are starting
kubectl get pods -n kube-system
```

At this point, only Master1 appears. The status will be `NotReady` because no CNI (pod network) plugin has been installed yet. That is expected.

STEP 7 — INSTALL THE POD NETWORK PLUGIN

Kubernetes does not include a built-in pod networking solution. A **Container Network Interface (CNI)** plugin must be installed before nodes can move to `Ready` status and before pods can communicate across nodes. Calico is one of the most widely used options for both lab and production environments.

INSTALL CALICO

```
kubectl apply -f https://docs.\
projectcalico.org/manifests/\
calico.yaml
```

Calico reads the `--pod-network-cidr` value you passed during `kubeadm init` (192.168.0.0/16) and configures itself accordingly.

VERIFY THE INSTALLATION

```
# Wait for Calico pods to become Running
kubectl get pods -n kube-system

# Master1 should now show Ready
kubectl get nodes
```

Calico deploys a DaemonSet that runs one pod per node. Once Master1's Calico pod is Running, Master1 transitions to `Ready`. Other nodes will become Ready after they join and their Calico pods start.

STEP 8 — JOIN MASTER2 AND MASTER3

With Master1 running and the network plugin installed, Master2 and Master3 can now join as additional control plane nodes. This is what makes the cluster truly highly available — each new master brings its own API Server, Scheduler, Controller Manager, and etcd instance, all automatically synchronized with Master1.

> Run the control-plane join command on Master2 first. Wait for it to complete and show Ready before joining Master3. Avoid joining both simultaneously — this can cause etcd membership race conditions.

```
sudo kubeadm join 10.0.0.100:6443 \
  --token abcdef.123456789 \
  --discovery-token-ca-cert-hash sha256:xxxxxxxxxxxxxxxx \
  --control-plane \
  --certificate-key abcd1234efgh
```

Run the identical command on both Master2 and Master3 (one at a time). After both complete, verify:

```
kubectl get nodes
# master1 Ready control-plane
# master2 Ready control-plane
# master3 Ready control-plane
```

WHAT HAPPENS INTERNALLY WHEN A MASTER JOINS

The `--control-plane` join process does far more than a worker join. It is a multi-phase process that copies PKI material, extends the etcd cluster, and starts all control plane components on the new node.

The certificate key passed with `--certificate-key` decrypts the PKI material stored in a Secret on the cluster. This eliminates the need to manually copy certificates between masters — `--upload-certs` during `kubeadm init` prepared this secret in advance.

CHAPTER 5: WORKER NODES

STEP 9 — JOIN WORKER NODES

Worker nodes are significantly simpler to join. They do not need certificates for the control plane, they do not run an API Server or etcd, and they do not need the `--control-plane` flag at all. The join command just needs the token and the CA hash to authenticate.

RUN ON WORKER1 AND WORKER2

```
sudo kubeadm join 10.0.0.100:6443 \
  --token abcdef.123456789 \
  --discovery-token-ca-cert-hash \
  sha256:xxxxxxxxxxxxxxxx
```

Both workers use the identical command. Run it on each node independently.

WHAT A WORKER NODE RUNS

KUBELET

Manages containers on the node, reports status to API Server every ~10s.

KUBE-PROXY

Maintains network rules for Service routing across the node.

CONTAINERD

Container runtime — pulls images and runs pod containers.

WORKER VS. MASTER JOIN COMMANDS — THE DIFFERENCE

Understanding exactly why these two commands differ is critical. The difference is not cosmetic — it reflects a fundamentally different role in the cluster.

Parameter	Worker Join	Master Join
Base command	`kubeadm join`	`kubeadm join`
`--token`	✅ Required	✅ Required
`--discovery-token-ca-cert-hash`	✅ Required	✅ Required
`--control-plane`	❌ Not used	✅ Required
`--certificate-key`	❌ Not used	✅ Required
Runs API Server	❌	✅
Runs etcd	❌	✅
Runs Scheduler	❌	✅
Runs kubelet	✅	✅
Runs kube-proxy	✅	✅

CHAPTER 6: VERIFICATION

STEP 10 — VERIFY THE COMPLETE CLUSTER

After all five nodes have joined, run a complete cluster health check from Master1. Every node should show `Ready` and the correct role label. System pods should all be running across all masters.

CHECK ALL NODES

```
kubectl get nodes -o wide

# Expected output:
NAME      STATUS   ROLES          AGE
master1   Ready    control-plane   10m
master2   Ready    control-plane   8m
master3   Ready    control-plane   6m
worker1   Ready    <none>          4m
worker2   Ready    <none>          3m
```

CHECK SYSTEM PODS

```
kubectl get pods -A

# You should see:
# kube-apiserver-master1
# kube-apiserver-master2
# kube-apiserver-master3
# kube-scheduler-master1
# etcd-master1, etcd-master2...
# kube-controller-manager-*
# calico-node-* (one per node)
# coredns-*
# kube-proxy-*
```

CHAPTER 7: ETCD DEEP DIVE

ETCD — THE REAL BRAIN OF KUBERNETES

etcd is a distributed key-value store that holds the entire state of a Kubernetes cluster. It is not just a component — it is the single source of truth. Every object in Kubernetes, from a Pod to a Secret to a Node registration, exists as a record in etcd.

WHAT ETCD STORES

Pods, Nodes, Deployments, Services, ConfigMaps, Secrets, RBAC rules, Namespaces, PersistentVolumes — every API object.

RAFT CONSENSUS

The three etcd instances use the RAFT algorithm to elect a leader and replicate every write to a majority of nodes before confirming success. This guarantees consistency.

QUORUM RULE

In a 3-node etcd cluster, at least 2 nodes must be healthy for the cluster to accept writes. This is why odd numbers (3, 5, 7) are always used — even failure tolerance.

HOW MASTERS STAY IN SYNC THROUGH ETCD

This is the question most people have when they first see a multi-master cluster: if there are three API Servers, how do they all see the same data? The answer is that they all read from and write to the same etcd cluster. There is no direct master-to-master communication.

WORKER JOINS

WRITE NODE

MASTERS WATCH

Every API Server runs a **watch** loop on etcd. The moment any object is created, updated, or deleted in etcd, all watching API Servers are notified simultaneously. This is what makes all three masters immediately consistent — no polling, no replication protocol between masters, just shared reads and writes to one etcd cluster.

CHAPTER 8: COMMUNICATION ARCHITECTURE

WORKER → MASTER VS. MASTER → WORKER

Traffic flows differently depending on direction. Understanding this distinction is fundamental to securing the cluster and configuring the right Security Group or firewall rules.

WORKER → MASTER (VIA LOAD BALANCER)

All worker-initiated communication goes through the Load Balancer to port **6443** on the API Server:

- Joining the cluster (`kubeadm join`)
- Kubelet heartbeats (~every 10 seconds)
- Pod status updates
- Node condition reporting

The worker never knows which master it's talking to. The LB handles routing transparently.

MASTER → WORKER (DIRECT CONNECTION)

All master-initiated communication goes **directly** to the kubelet on each worker node on port **10250**:

- Scheduling a pod to run on that node
- Streaming container logs
- Executing commands (`kubectl exec`)
- Forwarding ports (`kubectl port-forward`)

Masters know all worker IPs from etcd and connect directly — no Load Balancer involved.

COMPLETE PORT REFERENCE

Every inter-node communication in Kubernetes uses a specific port. These ports must be open in your firewall or AWS Security Groups for the cluster to function. The table below covers all critical ports for a 3-master + 2-worker cluster.

Port	Protocol	Purpose	Direction
6443	TCP	Kubernetes API Server	All nodes → Masters (via LB)
2379	TCP	etcd client API	API Server → etcd
2380	TCP	etcd peer communication	etcd ↔️ etcd (masters only)
10250	TCP	kubelet API	Masters → all worker nodes
10259	TCP	kube-scheduler	Internal master (localhost)
10257	TCP	kube-controller-manager	Internal master (localhost)
30000–32767	TCP	NodePort Services	External → Worker nodes

On AWS EC2: create two Security Groups — one for masters (allowing 6443, 2379, 2380 from within the VPC) and one for workers (allowing 10250 from masters, 30000-32767 from the internet). All nodes should allow all traffic within the Security Group for pod networking.

CHAPTER 9: LEADER ELECTION

WHY ONLY ONE SCHEDULER IS ACTIVE AT A TIME

All three masters run a Scheduler and a Controller Manager. But only **one** of each is actively making decisions at any given time. The others are running in standby. This prevents conflicts — if all schedulers tried to place pods simultaneously, the same pod could be scheduled to multiple workers.

Kubernetes solves this with **Leader Election**: a lock mechanism backed by etcd. Whichever master's Scheduler acquires the lock first becomes the leader. The lock is renewed periodically. If the leader fails to renew (because it crashed), the lock expires and the other masters compete to claim it.

LEADER ACQUIRES LOCK

One master writes a lease to etcd identifying itself as scheduler leader. This happens at startup.

LOCK IS RENEWED

The leader renews the lease every few seconds. Standby schedulers watch the lease and do nothing while it is held.

LEADER FAILS

If the lease expires (leader crashed), standby schedulers immediately race to claim it. First to write wins.

NEW LEADER ACTIVE

The new leader starts scheduling instantly. No human intervention required. The cluster heals itself.

CHECKING WHICH MASTER HOLDS THE LEADER LOCK

At any time, you can inspect which master is the active leader for both the Scheduler and the Controller Manager. This is useful for debugging and for understanding cluster behavior after a master failure.

CHECK SCHEDULER LEADER

```
kubectl get endpoints kube-scheduler \
  -n kube-system -o yaml
```

Look for the `holderIdentity` field in the `annotations` section. It will contain the name of the master node currently holding the scheduler lease.

CHECK CONTROLLER MANAGER LEADER

```
kubectl get endpoints \
  kube-controller-manager \
  -n kube-system -o yaml
```

The Scheduler and Controller Manager leaders are independent — they can (and often will) be on different masters. Both are elected separately through their own etcd leases.

All three masters always run kube-apiserver as **active** — there is no leader election for the API Server. The Load Balancer distributes API requests across all three simultaneously. Only the Scheduler and Controller Manager use leader election.

END-TO-END FLOW: FROM KUBECTL TO RUNNING CONTAINER

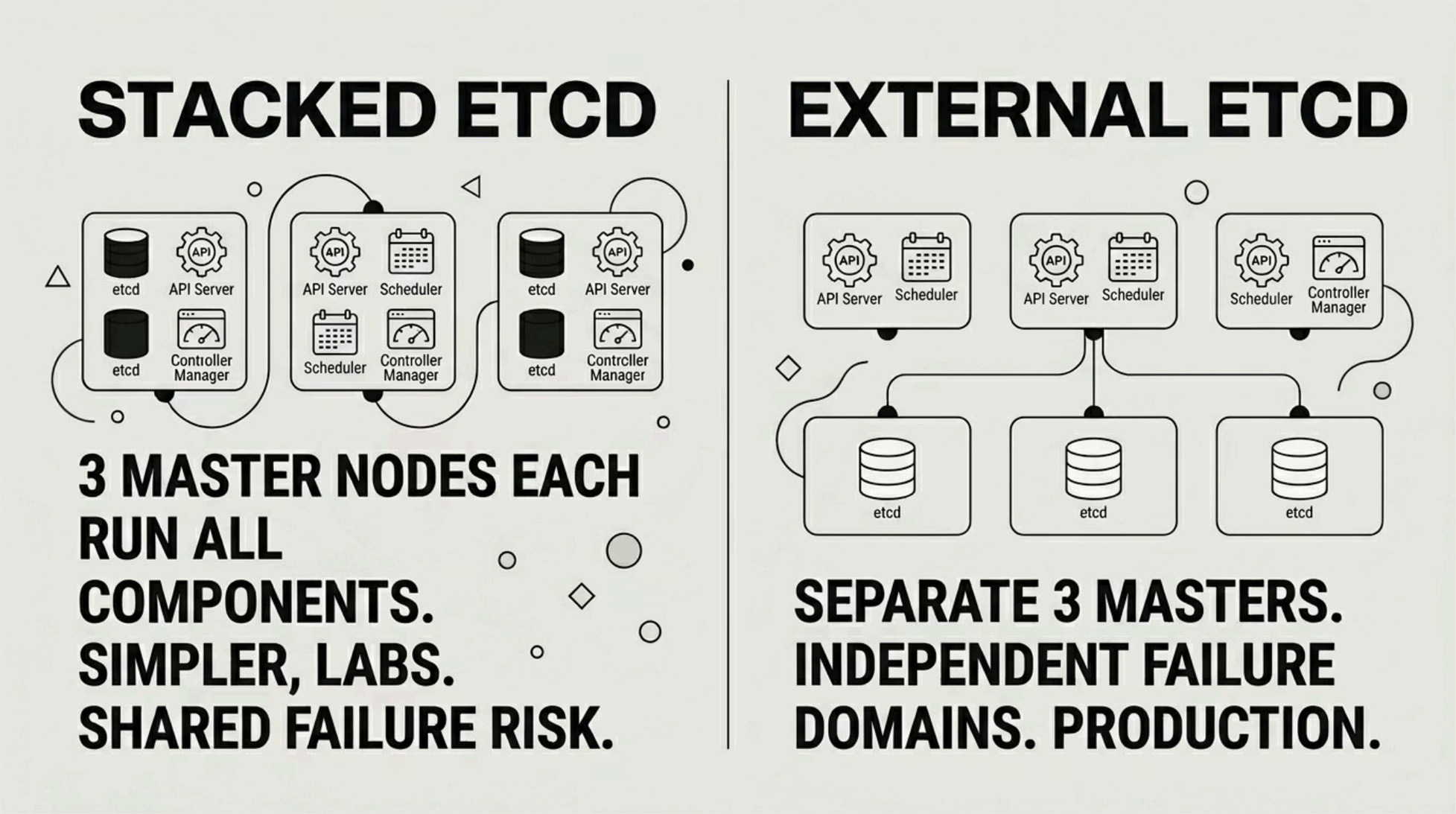
Understanding how a pod actually gets created — from the moment you run `kubectl apply` to the container running on a worker — reveals how every component we've built works together.

Notice that the Load Balancer is only involved in Step 1. Everything else is internal to the cluster. The Scheduler never talks to the worker directly — it writes its decision to etcd, and the kubelet on the worker continuously watches etcd (via the API Server) for pods assigned to its node.

CHAPTER 11: STACKED VS. EXTERNAL ETCD

TWO HA TOPOLOGY CHOICES

When building an HA Kubernetes cluster, there are two fundamentally different ways to deploy etcd. The choice affects resilience, operational complexity, and resource requirements. Understanding the difference matters before choosing your architecture.



Stacked etcd (the approach used throughout this guide) is the default for `kubeadm` and is appropriate for most clusters. **External etcd** is used when the cluster must tolerate a master node failure without any risk of losing etcd quorum — the etcd cluster survives independently.

CHAPTER 12: RECOVERING LOST TOKENS

HOW TO REGENERATE JOIN COMMANDS

Bootstrap tokens expire after 24 hours. If you need to add a new node after the original token has expired — or if you simply lost the output from `kubeadm init` — regenerate everything from Master1. No cluster restart is required.

REGENERATE WORKER JOIN COMMAND

```
# Generates a new token and prints
# the complete worker join command
kubeadm token create \
  --print-join-command
```

This creates a fresh 24-hour token and outputs the full `kubeadm join` command ready to copy and run on any worker node.

REGENERATE CA CERTIFICATE HASH

```
openssl x509 -pubkey \
  -in /etc/kubernetes/pki/ca.crt \
| openssl rsa -pubin -outform der \
  2>/dev/null \
| openssl dgst -sha256 -hex \
| sed 's/^.* //'
```

REGENERATE CERTIFICATE KEY (MASTERS)

```
# Uploads PKI material to the cluster
# and prints a new certificate-key
kubeadm init phase upload-certs \
  --upload-certs
```

Output example: `certificate key: 1234abcd5678efgh`

BUILD THE MASTER JOIN COMMAND

```
kubeadm join 10.0.0.100:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash \
  sha256:<hash> \
  --control-plane \
  --certificate-key <cert-key>
```

Combine the worker join token/hash with the certificate key to build the full master join command manually.

WITH VS. WITHOUT A LOAD BALANCER

Not every environment requires a dedicated load balancer. The decision affects whether the cluster is truly highly available at the API level, or just resilient at the data layer (etcd). Here is a clear comparison to guide the choice.

WITHOUT LOAD BALANCER (LAB ONLY)

Initialize with: `--apiserver-advertise-address=MASTER1_IP`

Workers connect to Master1 IP directly. etcd still forms a distributed cluster — data is shared. But if Master1 goes down, the API is unreachable even though Master2 and Master3 are healthy. Acceptable for learning; never acceptable for production.

WITH LOAD BALANCER (RECOMMENDED)

Initialize with: `--control-plane-endpoint "LB_IP:6443"`

Workers and operators connect to the LB virtual IP. If any master fails, the LB detects it (via health checks) and routes traffic to the remaining healthy masters. The cluster API remains available. This is what production requires.

3

MASTERS MINIMUM

Odd numbers only — 3, 5, or 7. Never 2 or 4. Even numbers cannot form a stable RAFT quorum.

1

KUBEADM INIT

Only ever run on one master. All others join. Running init twice destroys the cluster.

6443

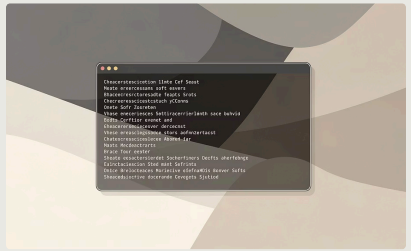
API PORT

The only port workers and operators ever need to reach on the Load Balancer.

CHAPTER 13: PRODUCTION CONSIDERATIONS

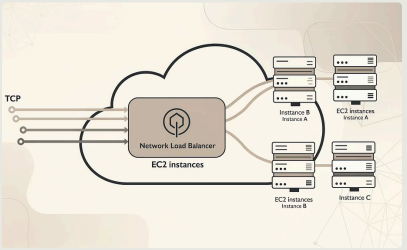
LOAD BALANCER OPTIONS BY ENVIRONMENT

The right Load Balancer depends entirely on where your cluster is running. Using the wrong type is one of the most common setup mistakes — particularly using ALB instead of NLB for the Kubernetes API Server.



HAPROXY (ON-PREM / VMS)

Simple, open source, runs on any Linux VM. Configured in minutes. The standard choice for kubeadm labs and bare-metal clusters. Add Keepalived for HA of the LB itself.



AWS NLB (EC2 CLUSTERS)

Use Network Load Balancer, not ALB. NLB supports TCP port 6443. ALB only supports HTTP/HTTPS and cannot forward the Kubernetes API protocol. Register all 3 masters as targets.

KEEPALIVED + HAPROXY (HA ON-PREM)

For production on-prem clusters, run two HAProxy instances with Keepalived managing a floating Virtual IP. If one HAProxy fails, the other claims the VIP automatically.

FINAL CHAPTER

COMPLETE IMPLEMENTATION SUMMARY

The complete build process follows 13 sequential steps. Every step matters — skipping or reordering them is the most common cause of cluster failures. The entire flow can be completed in under an hour on pre-provisioned VMs.

01

PREPARE ALL 5 NODES

Disable swap, enable kernel modules, set sysctl params on every machine.

02

INSTALL CONTAINERD

Container runtime on all nodes. Generate default config. Enable and start.

03

INSTALL KUBEADM, KUBELET, KUBECTL

Same version on all nodes. Lock with `apt-mark hold`.

04

CONFIGURE LOAD BALANCER

HAProxy or AWS NLB listening on TCP 6443, forwarding to all 3 master nodes.

05

KUBEADM INIT ON MASTER1

Use `--control-plane-endpoint` and `--upload-certs`. Save all join commands.

06

CONFIGURE KUBECTL ON MASTER1

Copy `admin.conf` to `~/.kube/config`. Verify with `kubectl get nodes`.

07

INSTALL CALICO CNI

Apply Calico manifest. Wait for Master1 to reach `Ready` state.

08

JOIN MASTER2 THEN MASTER3

Run control-plane join command with `--control-plane --certificate-key`. One at a time.

09

JOIN WORKER1 AND WORKER2

Run simple join command (no `--control-plane` flag) on each worker.

10

VERIFY CLUSTER HEALTH

`kubectl get nodes` and `kubectl get pods -A` — all nodes Ready, all system pods Running.

CLUSTER READY — KEY TAKEAWAYS

ONLY ONE KUBEADM INIT

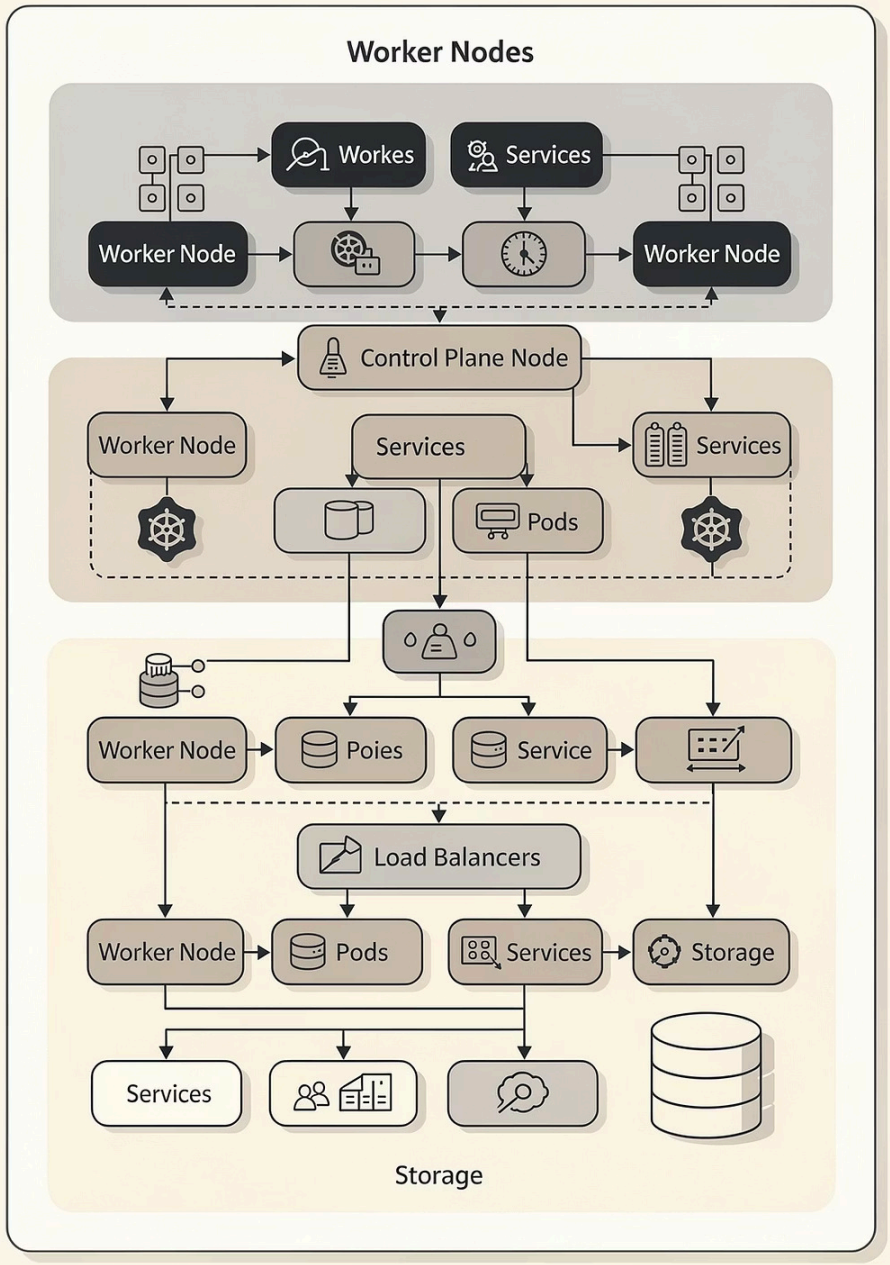
Master1 bootstraps the cluster. Master2 and Master3 join with `--control-plane`. Workers join with a plain join command.

ETCD SYNCHRONIZES MASTERS

Masters do not talk directly. They share state through the distributed etcd cluster. The Load Balancer only routes traffic — it does not synchronize data.

LEADER ELECTION FOR SCHEDULER

Only one Scheduler and one Controller Manager are active at a time, elected via an etcd lease. Failover is automatic and takes seconds.

API SERVER IS ALWAYS ACTIVE ON ALL MASTERS

Unlike Scheduler and Controller Manager, all three API Servers accept requests simultaneously. The LB distributes load across all of them.